# WEB APPLICATION SECURITY ASSESSMENT REPORT

Target Application: DVWA (Damn Vulnerable Web Application)

Testing Environment: Localhost via XAMPP

Security Level Tested: Low

Prepared by:

**Arkam Minnas**

**Email: arkamminnas4@gmail.com**

Cyber Security Internship Assessment

Treinetic (Pvt) Ltd

## Table of Contents


| | |
|---|---|
| 1. Introduction | Page 3 |
| 2. Methodology | Page 4 |
| 3. Tools Used | Page 5 |
| 4. Vulnerability Findings Summary | Page 6 |
| 5. Vulnerability 1 - SQL Injection | Page 7 |
| 6. Vulnerability 2 - Cross-Site Scripting (XSS) Reflected | Page 11 |
| 7. Vulnerability 3 - Cross-Site Scripting (XSS) Stored | Page 13 |
| 8. Vulnerability 4 - Brute Force / Authentication Issues | Page 15 |
| 9. Vulnerability 5 - IDOR | Page 17 |
| 10. Vulnerability 6 - Security Misconfiguration | Page 19 |
| 11. Risk Analysis Summary | Page 21 |
| 12. Mitigation and Fixes | Page 22 |
| 13. Conclusion | Page 23 |

# 1. Introduction

This report presents the findings of a security assessment performed on DVWA (Damn Vulnerable Web Application). DVWA is a web application that is intentionally built with security weaknesses. It is widely used by security professionals to practice finding and fixing vulnerabilities.

The goal of this assessment was to:

- Find and demonstrate real security vulnerabilities in the application
- Understand how each vulnerability works and what damage it can cause
- Suggest fixes to make the application more secure

This assessment was conducted as part of the Cyber Security Internship practical exam for Treinetic (Pvt) Ltd.

| | |
|---|---|
| **Application** | DVWA (Damn Vulnerable Web Application) |
| **Version / Stack** | PHP + MariaDB 10.4.32 via XAMPP |
| **Testing URL** | http://localhost/dvwa/ |
| **Security Level** | Low (intentionally set for demonstration) |
| **Tester** | Arkam Minnas |

## 2. Methodology

The testing followed a step-by-step approach to find vulnerabilities in the web application. Here is how the testing was done:

### Step 1 - Setup

DVWA was installed and run locally using XAMPP on a Windows machine. The security level was set to Low so that vulnerabilities could be clearly demonstrated.

### Step 2 - Exploring the Application

All pages and features of DVWA were explored including the login page, forms, and input fields to understand what the application does.

### Step 3 - Manual Testing

Each vulnerability was tested manually by entering special inputs (called payloads) into the input fields of the application to see how the application responds.

### Step 4 - Tool-Based Testing

Burp Suite was used to intercept and modify HTTP requests sent between the browser and the server. This helped demonstrate more advanced attacks like brute force.

### Step 5 - Collecting Evidence

Screenshots were taken at every important step to serve as proof of each vulnerability.

### Step 6 - Analysis and Reporting

Each vulnerability was analyzed, given a severity level, and documented with a real-world attack scenario and a suggested fix.

## 3. Tools Used

| | |
|---|---|
| **DVWA** | The vulnerable web application used as the target for testing |
| **XAMPP** | Used to run Apache web server and MySQL database locally on Windows |
| **Burp Suite Community** | Used to intercept, view, and modify HTTP requests between the browser and the server |
| **Burp Suite Intruder** | Used to automate brute force attack attempts on the login form |
| **Web Browser** | Used to interact with the application and observe outputs |

## 4. Vulnerability Findings Summary

The following vulnerabilities were identified during the assessment:

| # | Vulnerability | Severity | Description |
|---|---|---|---|
| 1 | **SQL Injection** | **Critical** | Attacker can read, modify, or delete all data in the database |
| 2 | **XSS (Reflected)** | **High** | Attacker can run malicious scripts in the victim's browser |
| 3 | **XSS (Stored)** | **High** | Malicious script is saved in the database and runs on every page load |
| 4 | **Brute Force / Auth Issues** | **High** | No login attempt limit allows password guessing attacks |
| 5 | **IDOR** | **Medium** | Users can access other users' private data by changing a number in the URL |
| 6 | **Security Misconfiguration** | **Medium** | Server reveals sensitive technical information to attackers |

# 5. Vulnerability 1 - SQL Injection

| | |
|---|---|
| **Location** | http://localhost/dvwa/vulnerabilities/sqli/ |
| **Severity** | CRITICAL |
| **Type** | SQL Injection (Union-Based) |

## What is it?

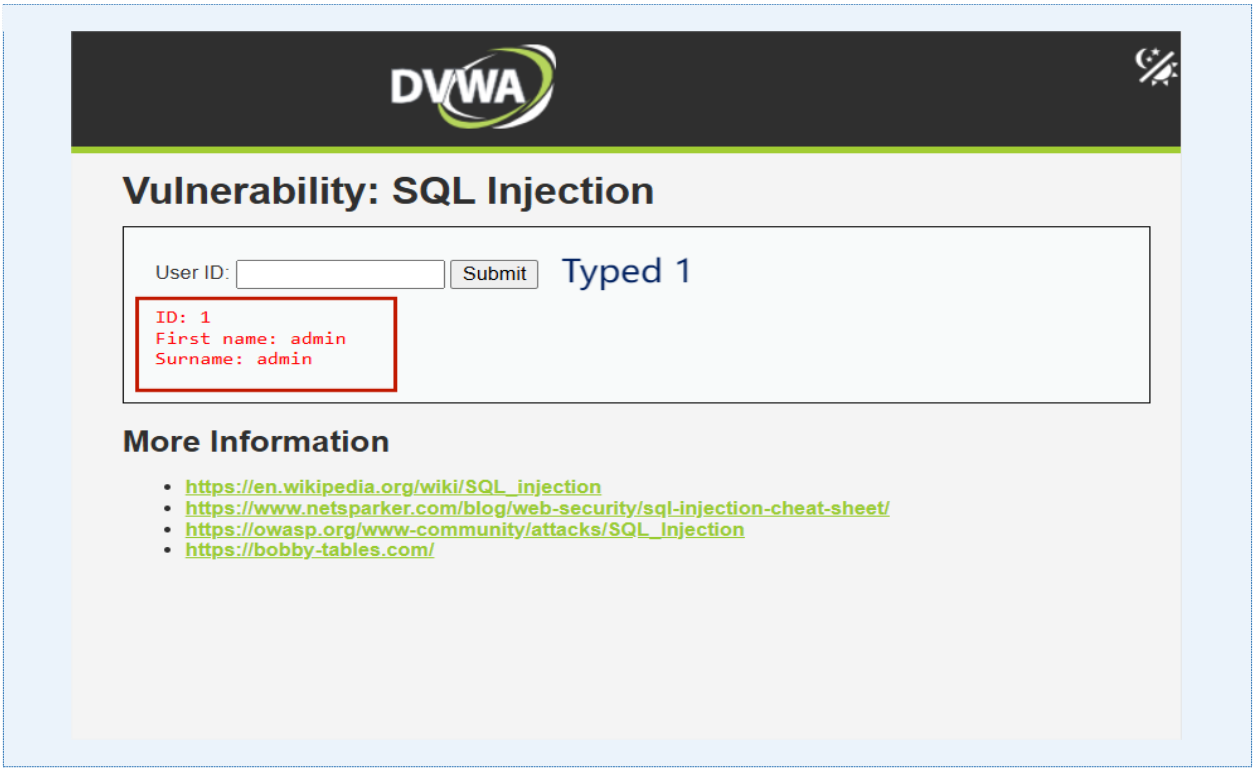
SQL Injection happens when an attacker types special characters into an input field that tricks the database into running unintended commands. The application does not properly check what the user types before passing it to the database query.

## What was done?

The User ID input field was tested with special SQL code. The application directly put the user's input into the SQL query without checking it, making it vulnerable.

### Test 1 - Normal Input (User ID: 1)

Typing '1' shows the normal expected result - the admin user's first name and surname.

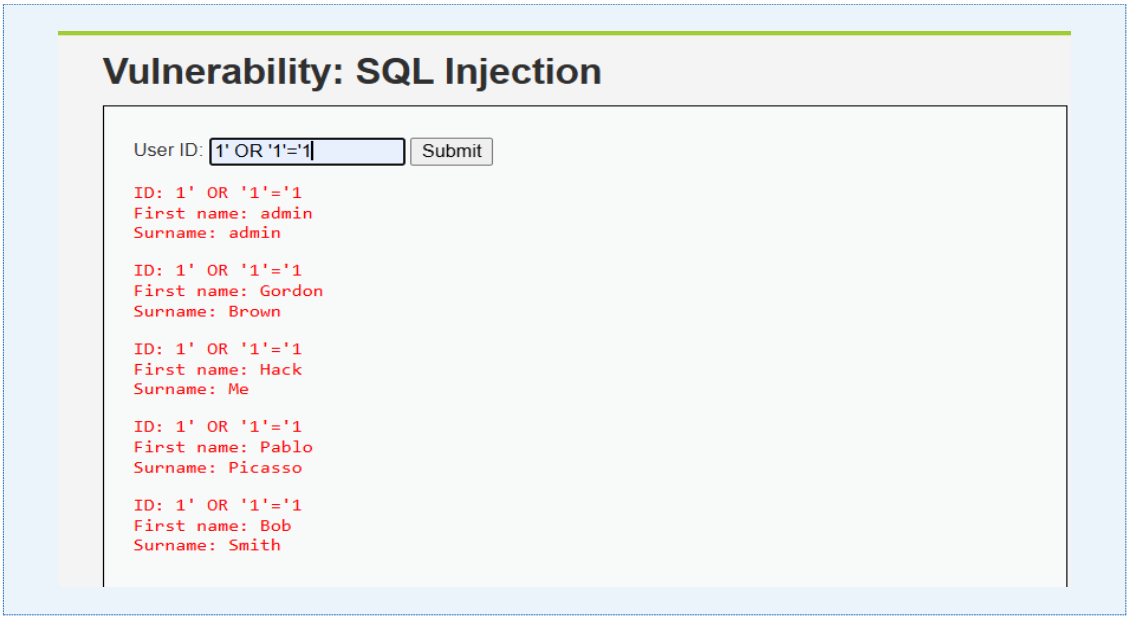


### Test 2 - Dump All Users

Payload used:

```
1' OR '1'='1
```

This payload tricks the query into returning ALL users in the database because '1'='1' is always true. The names of all 5 users were revealed.

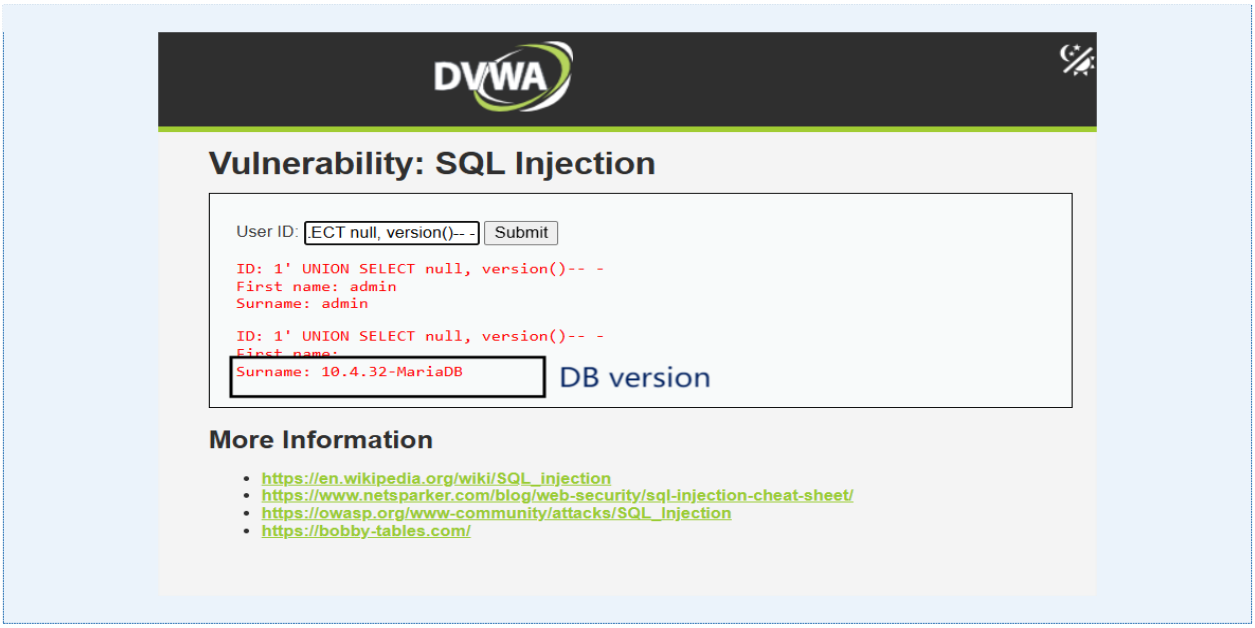



### Test 3 - Extract Database Version

Payload used:

```
1' UNION SELECT null, version()-- -
```

This revealed that the database is running MariaDB version 10.4.32. Knowing the database version helps attackers find known security weaknesses in that version.

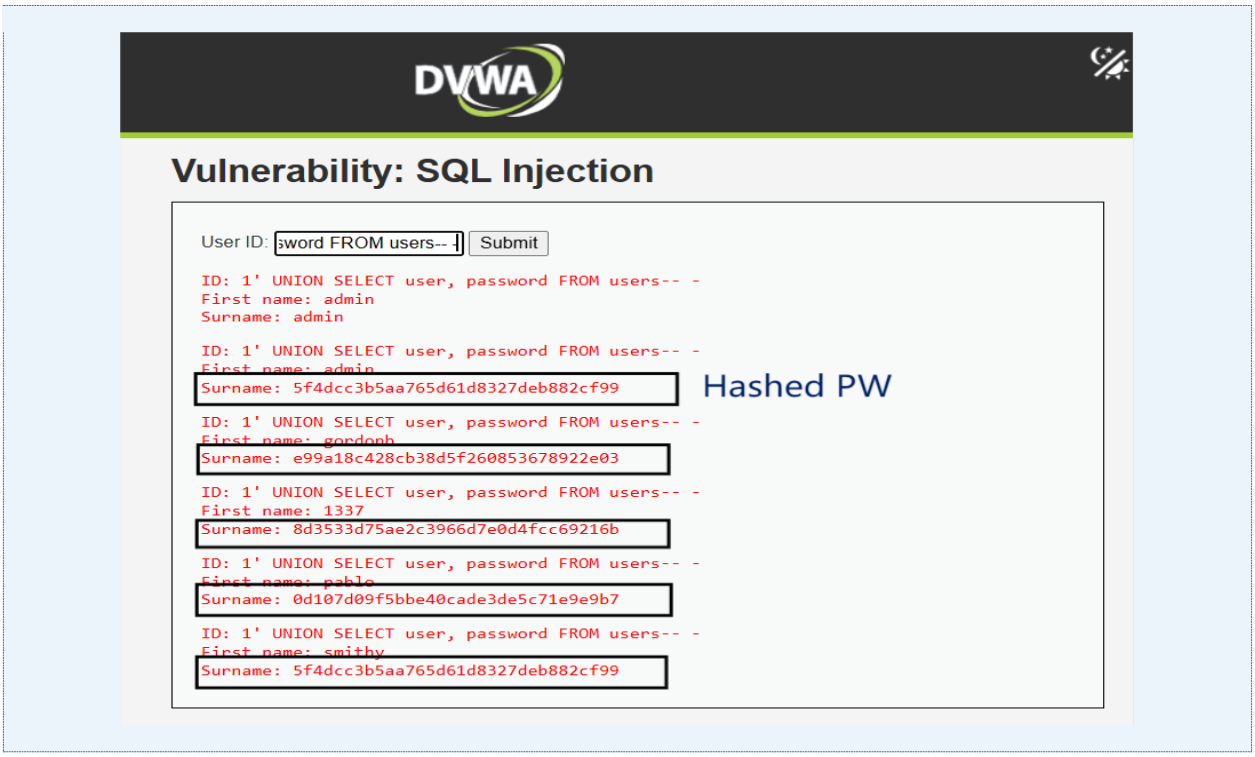



## Test 4 - Dump All Passwords

Payload used:

```
1' UNION SELECT user, password FROM users-- -
```

This dumped all usernames and their hashed passwords from the database. An attacker can use tools to crack these password hashes.



## Real World Attack Scenario

Imagine a real banking website has this vulnerability. An attacker could type a SQL payload into the login form and extract all customer names, emails, and passwords from the database. They could then use the stolen passwords to log into the bank accounts of all customers, causing massive financial and personal harm.

## Fix

Use Prepared Statements (also called Parameterized Queries). This means the SQL query is written first with placeholders, and the user's input is added separately. This way, the input can never change the structure of the SQL query.

Example fix in PHP:

```
$stmt = $db->prepare("SELECT * FROM users WHERE id = ?");
$stmt->bind_param("s", $id);
$stmt->execute();
```

# 6. Vulnerability 2 - XSS Reflected

| | |
|---|---|
| **Location** | http://localhost/dvwa/vulnerabilities/xss_r/ |
| **Severity** | HIGH |
| **Type** | Cross-Site Scripting - Reflected |

## What is it?

Reflected XSS happens when an attacker puts malicious JavaScript code into an input field and the website immediately shows that code back in the page without checking it. The browser then runs the script thinking it came from the trusted website.

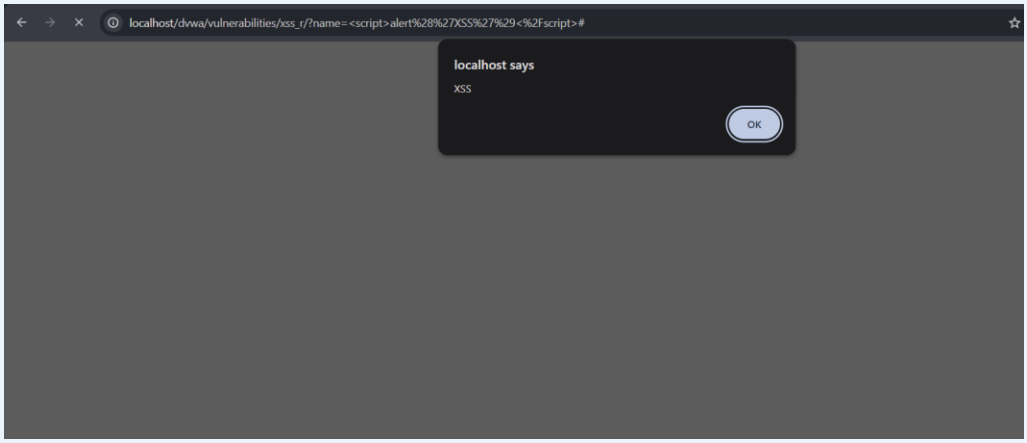
## What was done?

### Test 1 - Basic Alert Popup

Payload used:

```
<script>alert('XSS')</script>
```

This was entered in the 'What's your name?' field. The page ran the script and showed a popup alert box, proving the site will execute any JavaScript entered by the user.

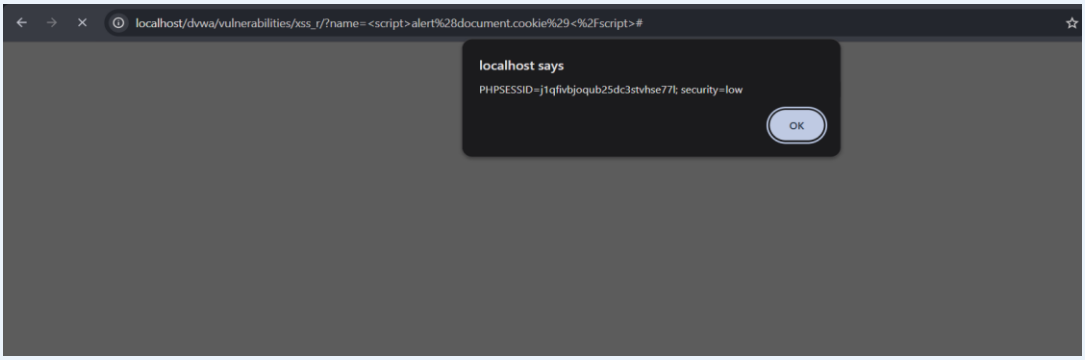



### Test 2 - Steal Session Cookie

Payload used:

```
<script>alert(document.cookie)</script>
```

This showed the user's session cookie inside a popup. In a real attack, instead of showing the cookie, the attacker would secretly send it to their own server and use it to hijack the victim's logged-in session.




## Real World Attack Scenario

An attacker sends a link to a victim that contains a malicious XSS payload in the URL. When the victim clicks the link and opens the page, their session cookie is secretly sent to the attacker's server. The attacker uses this cookie to log into the website as the victim without needing their password.

## Fix

- Encode all output - convert special characters like < and > into safe HTML entities like < and > before displaying them on the page
- Use Content Security Policy (CSP) headers to block inline scripts from running

Example fix in PHP:

```
echo htmlspecialchars($name, ENT_QUOTES, "UTF-8");
```

# 7. Vulnerability 3 - XSS Stored

| | |
|---|---|
| **Location** | http://localhost/dvwa/vulnerabilities/xss_s/ |
| **Severity** | HIGH |
| **Type** | Cross-Site Scripting - Stored |

## What is it?

Stored XSS is more dangerous than Reflected XSS. Instead of the script running only once from a URL, the malicious script is saved into the database. Every time any user visits the page, the script automatically runs in their browser.

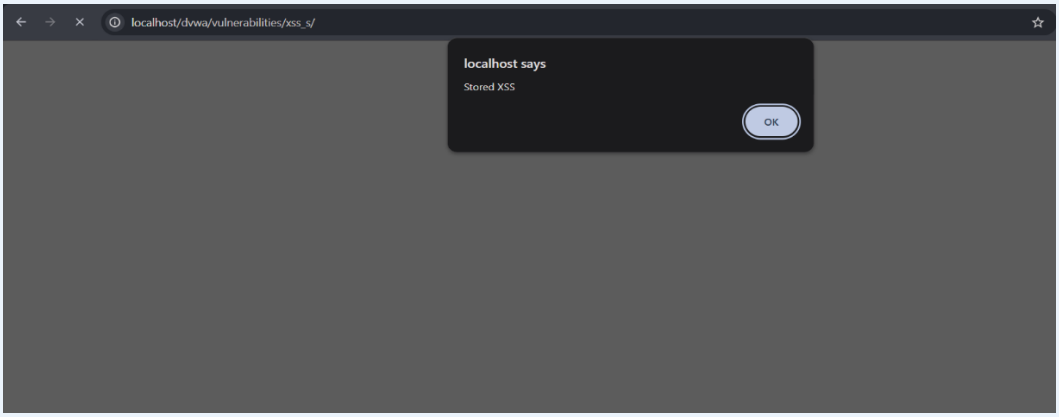
## What was done?

### Test 1 - Basic Stored Script

In the guestbook form:

- Name field: Hacker
- Message field: <script>alert('Stored XSS')</script>

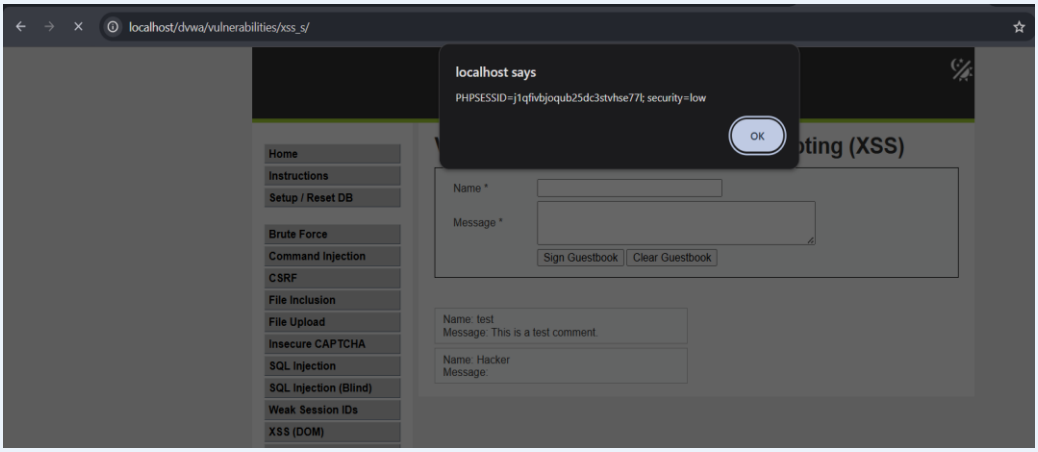
After clicking 'Sign Guestbook', every time the guestbook page is loaded by any user, the alert popup fires automatically. The script is permanently saved in the database.




### Test 2 - Cookie Theft via Stored XSS

- Message field: <script>alert(document.cookie)</script>

This stored script shows every visitor's session cookie when they visit the page.




## Real World Attack Scenario

An attacker posts a comment on a popular website with a hidden script inside it. Every single user who reads that comment has their session cookie stolen and sent to the attacker. The attacker can then take over thousands of accounts without the victims knowing anything happened.

## Fix

- Sanitize all user input before saving it to the database
- Encode all content before displaying it on the page
- Use a library like HTMLPurifier in PHP to clean user-submitted content

```
$message = htmlspecialchars($message, ENT_QUOTES, "UTF-8");
```

# 8. Vulnerability 4 - Brute Force / Authentication Issues

| | |
|---|---|
| **Location** | http://localhost/dvwa/vulnerabilities/brute/ |
| **Severity** | HIGH |
| **Type** | Authentication Weakness - No Rate Limiting |

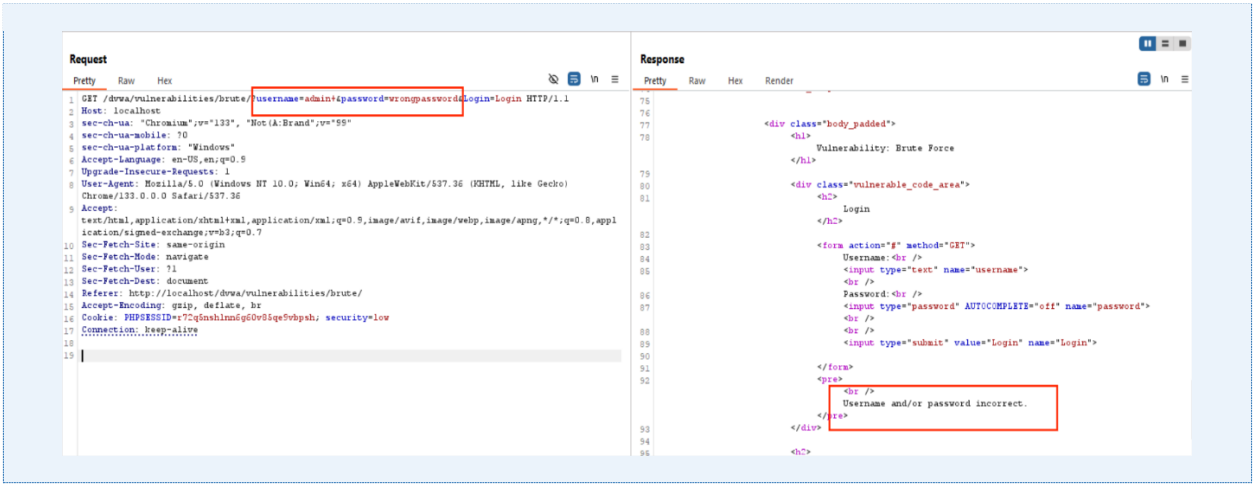
## What is it?

Brute force is when an attacker tries many different passwords one after another until they find the correct one. If a website does not limit how many times someone can try to log in, an attacker can use automated tools to try thousands of passwords very quickly.

## What was done?

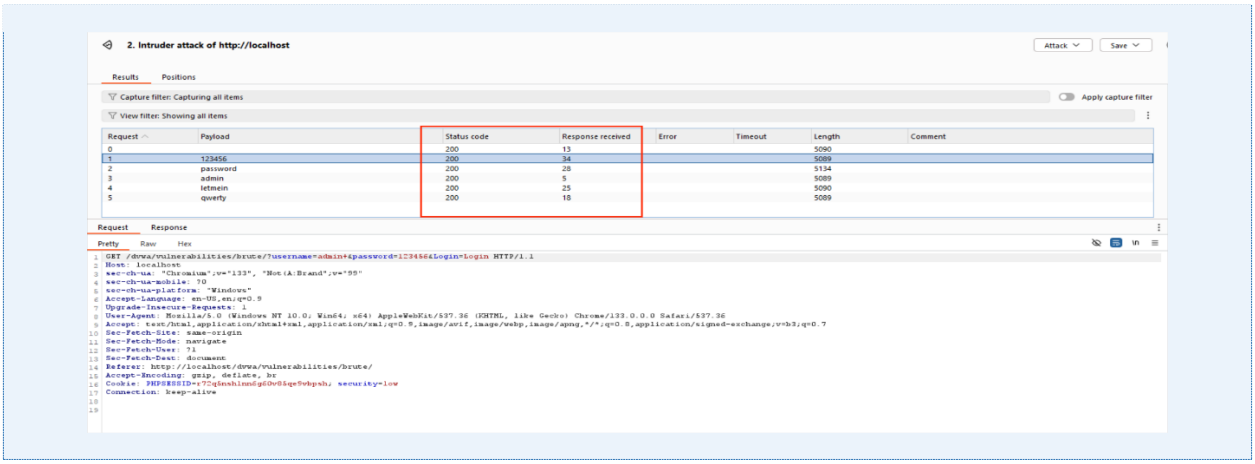
### Step 1 - Intercept Login Request with Burp Suite

The login form was submitted with wrong credentials (admin / wrongpassword). Burp Suite intercepted the HTTP GET request, which showed the username and password being sent in plain text in the URL.




### Step 2 - Brute Force Attack with Burp Intruder

The request was sent to Burp Suite's Intruder tool. The password field was marked as the attack position and a list of common passwords was used. The attack ran 5 attempts automatically.




Key findings from the Intruder attack:

- All 5 requests returned HTTP 200 - the server never blocked or slowed down the attempts
- The request with 'password' returned a response length of 5134, different from others at 5089, indicating a successful login
- No CAPTCHA, no account lockout, no rate limiting was observed

## Additional Issues Found

- Credentials are sent via GET request - passwords appear in the URL and browser history
- No CSRF token present - the form has no protection against cross-site request forgery
- Default credentials admin/password were never changed

## Real World Attack Scenario

An attacker uses an automated tool to try 10,000 common passwords against an admin login page. Because there is no lockout after failed attempts, the tool runs through all passwords in minutes and successfully logs in as admin, gaining full control of the website.

## Fix

- Lock an account for 15-30 minutes after 5 failed login attempts
- Add CAPTCHA to the login form to block automated tools
- Use POST instead of GET for login forms so passwords don't appear in URLs
- Require strong passwords and never use default credentials in production

```
// Example: Track failed attempts in session
if ($failed_attempts >= 5) { block_login_for(30); }
```

# 9. Vulnerability 5 - IDOR (Insecure Direct Object Reference)

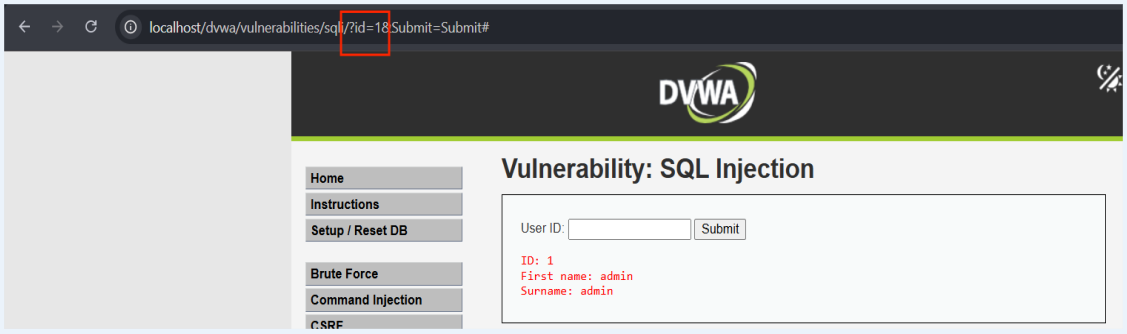
| | |
|---|---|
| **Location** | http://localhost/dvwa/vulnerabilities/sqli/ |
| **Severity** | MEDIUM |
| **Type** | Broken Access Control - IDOR |

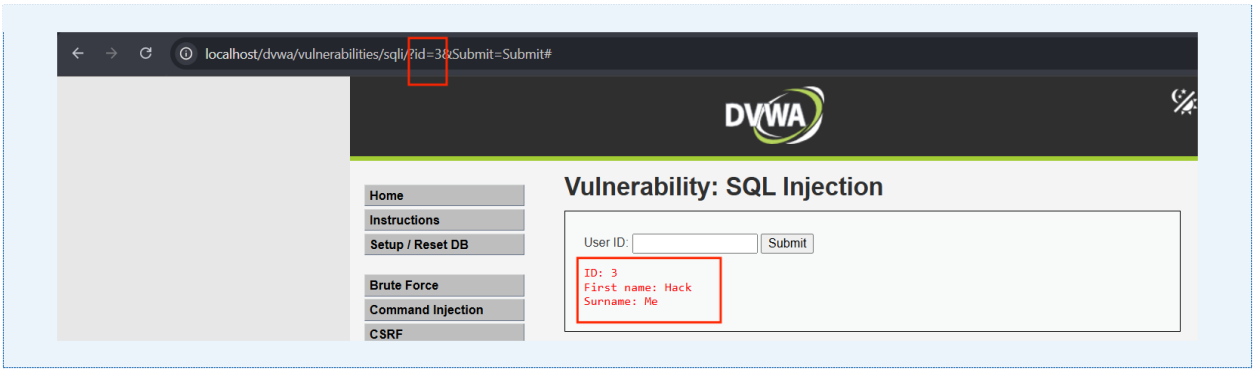
## What is it?

IDOR happens when a website lets users access other people's data just by changing a number or ID in the input or URL. The website does not check if the logged-in user is allowed to see that particular data.

## What was done?

By simply changing the User ID number in the input field (1, 2, 3, 4, 5), the personal information of each user in the database was accessible. There was no check to confirm whether the logged-in user had permission to view another user's data.






## Real World Attack Scenario

On an online shopping website, after placing an order, a user sees their order details at a URL like /orders/1001. The attacker changes 1001 to 1002, 1003, and so on. Because there is no access control check, they can view the name, address, and purchase history of every customer who ever placed an order.

## Fix

- Always check on the server side that the logged-in user owns the data they are requesting
- Never rely only on the client side (browser) to control access
- Use session-based checks: compare the requested user ID against the session's user ID

```
// Example fix in PHP
if ($_SESSION["user_id"] !== $requested_id) {
    die("Access Denied");
}
```

# 10. Vulnerability 6 - Security Misconfiguration

| | |
|---|---|
| **Location** | Multiple locations across DVWA |
| **Severity** | MEDIUM |
| **Type** | Security Misconfiguration |

## What is it?

Security misconfiguration happens when a web server or application is not set up correctly. This can give attackers useful information about the system or allow them to access things they should not be able to.

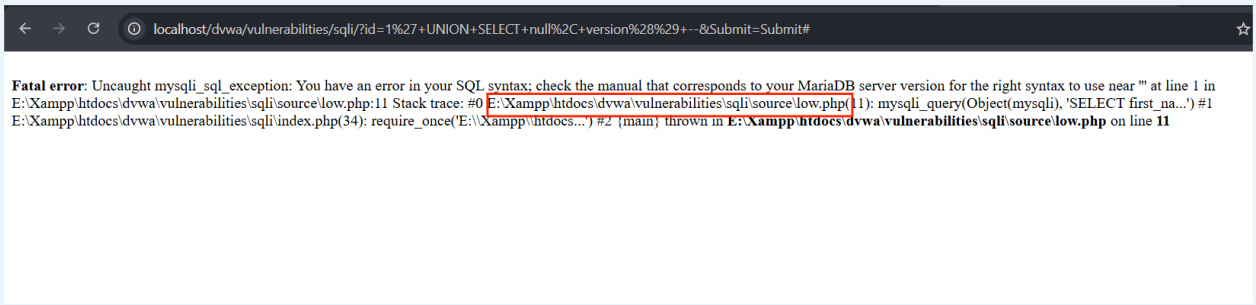
## What was found?

### Issue 1 - Detailed Error Messages Exposed

When a bad SQL payload was entered, the application showed a full error message including:

- The full file path on the server: E:\Xampp\htdocs\dvwa\vulnerabilities\sqli\source\low.php
- The exact SQL query that was running
- The database type and version

This gives attackers a map of the server's file structure and technology stack.

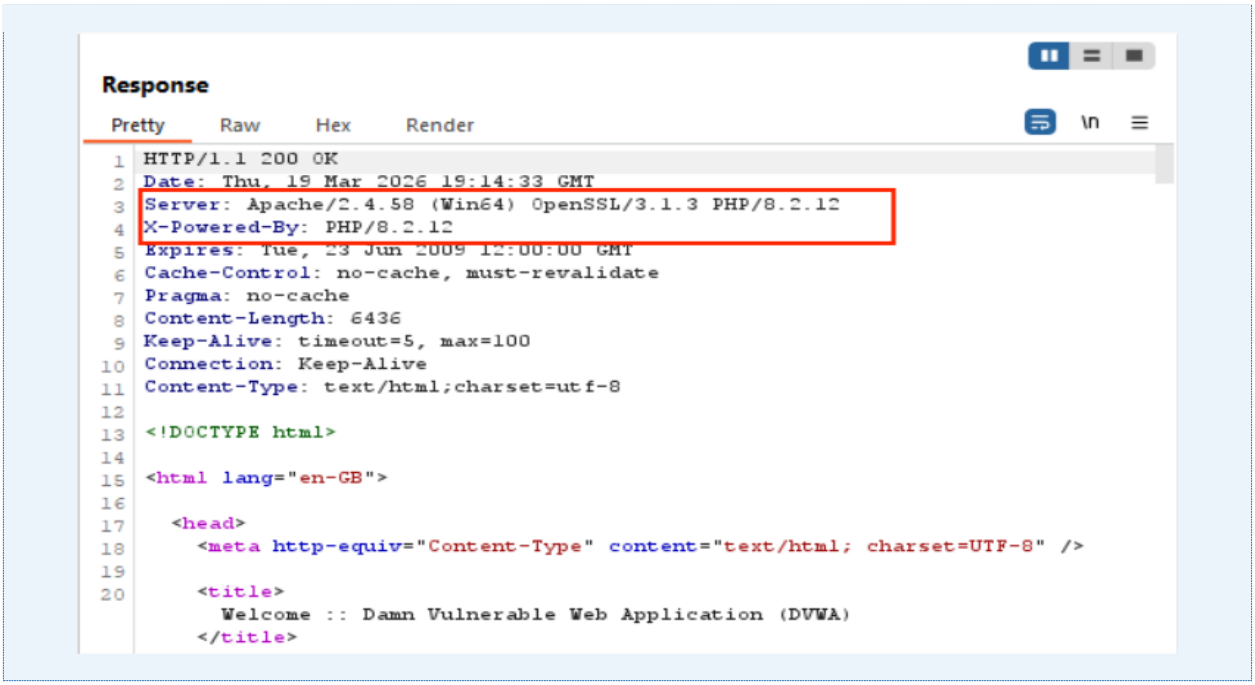



### Issue 2 - Server Headers Reveal Technology

The HTTP response headers from the server revealed:

- The web server software and version (Apache)
- The PHP version being used (X-Powered-By header)

Attackers can use this information to look up known vulnerabilities in those specific versions.




### Issue 3 - Default Credentials in Use

The application was accessible using the default username 'admin' and password 'password'. These credentials are publicly known and should always be changed before deployment.

## Real World Attack Scenario

An attacker visits a login page and intentionally triggers an error. The detailed error message reveals that the server is running an old version of PHP with a known security flaw. The attacker searches online for an exploit for that version and uses it to take full control of the web server.

## Fix

- Turn off detailed error messages on production servers - only show generic error messages to users
- Log errors privately on the server instead of displaying them
- Remove or hide the X-Powered-By header to not reveal the PHP version
- Always change default usernames and passwords before going live

```
// PHP - disable error display in production
ini_set('display_errors', 0);
error_reporting(0);
```

## 11. Risk Analysis Summary

Each vulnerability was given a severity rating based on OWASP standards. The rating considers how easy the attack is to perform and how much damage it can cause.

| Vulnerability | Severity | Impact | Real-World Risk |
|---|---|---|---|
| **SQL Injection** | **Critical** | Full database access | All user data stolen, accounts compromised |
| **XSS Reflected** | **High** | Session hijacking | Attacker logs in as victim without password |
| **XSS Stored** | **High** | Mass session theft | Thousands of users affected at once |
| **Brute Force** | **High** | Admin account takeover | Full site control gained via password guessing |
| **IDOR** | **Medium** | Unauthorized data access | Any user's private info viewed by others |
| **Sec. Misconfig** | **Medium** | Info leakage to attackers | Attackers learn system internals to plan attacks |

## 12. Mitigation and Fixes Summary

The following fixes are recommended to address all vulnerabilities found in this assessment:

| Vulnerability | Root Cause | Recommended Fix |
|---|---|---|
| **SQL Injection** | User input directly added into SQL query without any checks | Use prepared statements and parameterized queries. Never build SQL queries using user input directly. |
| **XSS Reflected** | User input shown on page without encoding special characters | Use htmlspecialchars() to encode output. Add Content-Security-Policy HTTP header. |
| **XSS Stored** | User input saved to database and shown on page without sanitization | Sanitize input before saving. Encode output before displaying. Use HTMLPurifier library. |
| **Brute Force** | No limit on failed login attempts. No CAPTCHA. | Add account lockout after 5 failed attempts. Add CAPTCHA. Use POST for login forms. |
| **IDOR** | No server-side check to verify if user owns the requested data | Always verify on the server that the session user matches the requested data owner. |
| **Sec. Misconfig** | Detailed errors shown to users. Default credentials used. Server headers reveal version info. | Disable error display in production. Remove X-Powered-By header. Change default credentials. |

## 13. Conclusion

This security assessment of DVWA successfully identified and demonstrated six web application vulnerabilities. These vulnerabilities represent some of the most common and dangerous security issues found in real web applications today, as listed in the OWASP Top 10.

The most critical finding was SQL Injection, which allowed complete access to the application's database including all user credentials. This type of vulnerability, if found in a real application, could result in a major data breach affecting thousands of users.

The XSS vulnerabilities (both Reflected and Stored) demonstrated how an attacker can steal user sessions and take over accounts without ever knowing the victim's password. The Stored XSS variant is especially dangerous because it affects every single visitor to the page.

The Brute Force vulnerability showed that without proper login protection, automated tools can crack passwords in minutes. The IDOR vulnerability highlighted how simple it is to access other users' data when access controls are not properly implemented.

Finally, Security Misconfiguration demonstrated that even small mistakes in server setup - like showing detailed error messages or using default credentials - can give attackers the information they need to plan and execute more sophisticated attacks.

All of the vulnerabilities found have well-known and straightforward fixes. Implementing the recommended mitigations would significantly improve the security of any web application. The key takeaways are:

- Always validate and sanitize user input - never trust what a user types
- Use prepared statements for all database queries
- Implement proper access controls and verify them on the server side
- Protect login forms with rate limiting and CAPTCHA
- Hide sensitive server information from error messages and HTTP headers
- Never deploy applications with default credentials

Security is not a one-time task but an ongoing process. Regular security testing and code reviews are essential to keeping web applications safe.

**Prepared by: Arkam Minnas**

Cyber Security Internship Assessment - Treinetic (Pvt) Ltd